

M1 DEVFLSTK - CI/CD

TaskFlow API

Pipeline Jenkins CI/CD

Rapport final - Projet DevOps Docker, MongoDB, Nginx et Jenkins

Node.js + Express

MongoDB

Docker Compose

Jenkins Declarative Pipeline

Couverture 93.05%

Binôme

Arthur Courchay
Mathias Bouclon

Date

22 juin 2026

Repository GitHub

https://github.com/Arthurccy/Bouclon_Mathias_Courchay-Arthur_TP-Final-CI-CD

Livrable

API REST conteneurisée, pipeline Jenkins, rapport PDF et preuves d'exécution.

Projet réalisé dans le cadre du module CI/CD - Caplogy / YNOV.

1. Synthèse du projet

TaskFlow API est une API REST de gestion de tâches créée spécifiquement pour le projet final CI/CD. L'objectif était de partir d'une application simple, puis de l'industrialiser comme un livrable professionnel: tests automatisés, lint strict, conteneurisation, orchestration, reverse proxy et pipeline Jenkins complet.

Résultat principal: le pipeline Jenkins exécute les 6 stages attendus (Checkout, Install, Lint, Test, Build Docker, Deploy) et déploie l'application via Docker Compose.

Objectifs couverts

Zone	Réalisation
Application	API Express avec 6 routes REST, modèle Mongoose, validation métier.
Qualité	ESLint strict, 14 tests Jest, couverture globale 93.05%.
Infrastructure	Dockerfile multi-stage, Docker Compose, MongoDB, Nginx, réseau dédié.
CI/CD	Jenkins en service Docker, credentials, build Docker tagué, déploiement automatisé.
Sécurité	Aucun secret dans le code, .env ignoré, MongoDB non exposé.

Sommaire

1. Architecture globale
2. Application TaskFlow API
3. Docker, Compose et Nginx
4. Pipeline Jenkins
5. Preuves d'exécution
6. Build rouge intentionnel
7. Répartition du binôme
8. Difficultés rencontrées

2. Architecture globale

L'architecture repose sur un flux simple: l'utilisateur passe par Nginx, qui relaie les requêtes vers l'API Express. L'API communique avec MongoDB sur un réseau Docker interne. Jenkins orchestre la chaîne CI/CD à partir du repository GitHub.

```
GitHub
|
| push / webhook
v
Jenkins :8080
|
| Checkout -> Install -> Lint -> Test -> Build Docker -> Deploy
v
Docker Compose
```

```
Utilisateur -> localhost:80 -> Nginx -> API Express :5000 -> MongoDB :27017  
réseau interne: taskflow-network
```

Flux applicatif

- Nginx expose le port 80.
- L'API reste accessible uniquement dans Docker sur le port 5000.
- MongoDB reste interne au réseau Docker.
- Les routes /health et /api/tasks sont servies via Nginx.

Flux CI/CD

- Jenkins récupère le code depuis GitHub.
- Le pipeline bloque en cas d'erreur ESLint ou Jest.
- L'image API est taguée en latest et build-N.
- Le déploiement est effectué par `docker compose up`.

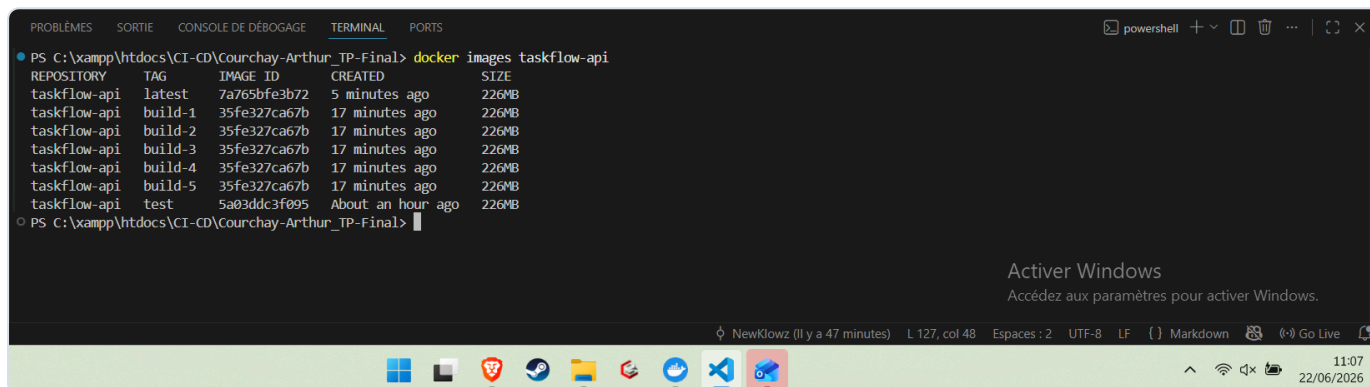


Figure 1 - Conteneurs Docker TaskFlow avec Jenkins, Nginx, API et MongoDB.

3. Application TaskFlow API

L'API a été développée en Node.js avec Express. Elle permet de gérer des tâches avec un modèle volontairement simple, mais suffisant pour démontrer une chaîne CI/CD complète et testable.

Modèle de données

Champ	Type	Règle
title	String	Obligatoire, trim automatique.
description	String	Optionnel, valeur par défaut vide.
status	String	Enum: todo, in-progress, done.
createdAt	Date	Généré automatiquement.

Routes REST

Méthode	Route	Description
GET	/health	Retourne l'état de l'API, l'uptime et la version.
GET	/api/tasks	Liste les tâches.
POST	/api/tasks	Crée une tâche.
GET	/api/tasks/:id	Retourne une tâche par identifiant.
PUT	/api/tasks/:id	Met à jour le statut.
DELETE	/api/tasks/:id	Supprime une tâche.

```
PS C:\xampp\htdocs\CI-CD\Courchay-Arthur_TP-Final> curl.exe http://localhost/health
>> curl.exe http://localhost/api/tasks
{"status":"ok","uptime":716.197717968,"version":"1.0.0"}[]
PS C:\xampp\htdocs\CI-CD\Courchay-Arthur_TP-Final> █
```

Figure 2 - Vérification des endpoints `/health` et `/api/tasks`.

4. Tests et qualité logicielle

Les tests Jest couvrent les comportements demandés dans les consignes: health-check, validation du titre, liste des tâches et scénario d'intégration création/récupération. Des cas complémentaires ont été ajoutés pour couvrir les erreurs et les mises à jour.

La suite contient 14 tests verts. La couverture globale est de 93.05% sur les statements, 90.9% sur les branches et 100% sur les fonctions.

Approche de test

- Tests HTTP réalisés avec supertest.
- MongoDB réel non requis pendant les tests.
- Modèle Task mocké avec Jest.
- Pipeline bloquant si un test échoue.

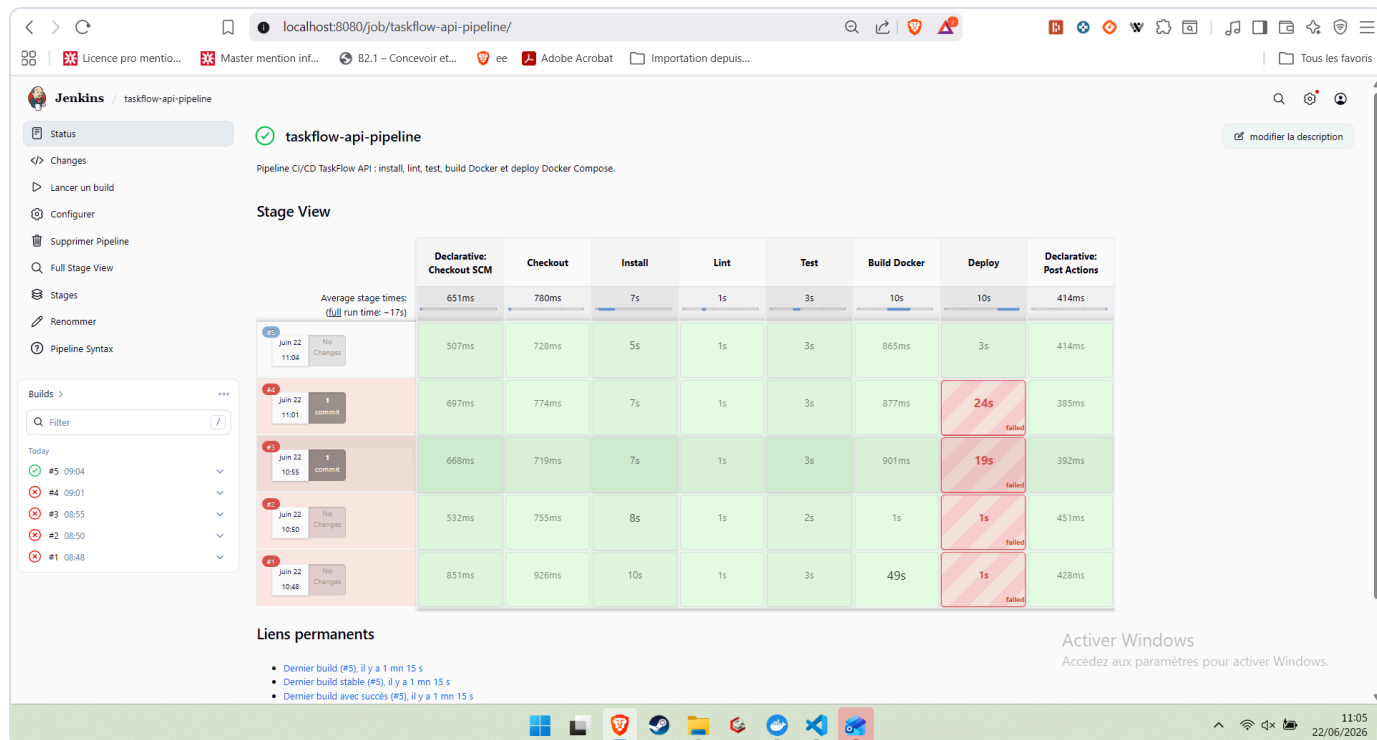


Figure 3 - Exécution Jest et couverture de tests.

5. Docker, Compose et Nginx

L'application est conteneurisée avec un Dockerfile multi-stage. Le stage builder installe les dépendances nécessaires au build, tandis que le stage production installe uniquement les dépendances de production et exécute l'application avec un utilisateur non-root.

Choix Docker

Élément	Décision	Justification
Image	node:18-alpine	Base légère conforme aux consignes.
Multi-stage	builder + production	Sépare devDependencies et runtime.
Sécurité	Utilisateur taskflow	Évite l'exécution root en production.
Healthcheck	/health	Permet à Compose et Jenkins de vérifier l'API.

Orchestration

- mongodb: base de données avec volume nommé mongodb_data.
- api: service Express, dépend de MongoDB healthy.
- nginx: reverse proxy exposé sur le port 80.
- jenkins: service CI/CD exposé sur le port 8080.

```
PS C:\xampp\htdocs\CI-CD\Courchay-Arthur_TP-Final> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
cb3ac73060cc   taskflow-jenkins:local              "/usr/bin/tini -- /u..." 6 minutes ago  Up 6 minutes  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
501e59656feb   nginx:1.27-alpine                  "/docker-entrypoint..." 13 minutes ago  Up 13 minutes (healthy)  0.0.0.0:80->80/tcp, [::]:80->80/tcp
8cfe036b4d19   taskflow-api:latest                "docker-entrypoint.s..." 13 minutes ago  Up 13 minutes (healthy)  5000/tcp
4251596d5fda   mongo:6                             "docker-entrypoint.s..." 13 minutes ago  Up 13 minutes (healthy)  27017/tcp
taskflow-mongodb
PS C:\xampp\htdocs\CI-CD\Courchay-Arthur_TP-Final>
```

Figure 4 - État Docker après déploiement de la stack.

6. Pipeline Jenkins

Le pipeline est écrit en syntaxe Declarative et versionné à la racine du dépôt dans le fichier `Jenkinsfile`. Il automatise la validation du code et le déploiement de la stack.

Stage	Commande principale	Rôle
Checkout	checkout scm	Récupérer le code GitHub.
Install	npm ci	Installer les dépendances reproductiblement.
Lint	npm run lint	Bloquer le pipeline en cas de violation ESLint.
Test	npm test	Lancer Jest et afficher la couverture.
Build Docker	docker build	Construire latest et build-N.
Deploy	docker compose up	Déployer MongoDB, API et Nginx.

Le secret `MONGO_URI` est injecté via Jenkins Credentials avec l'ID `mongo-uri`. Aucun secret n'est présent dans le `Jenkinsfile` ou dans le dépôt.

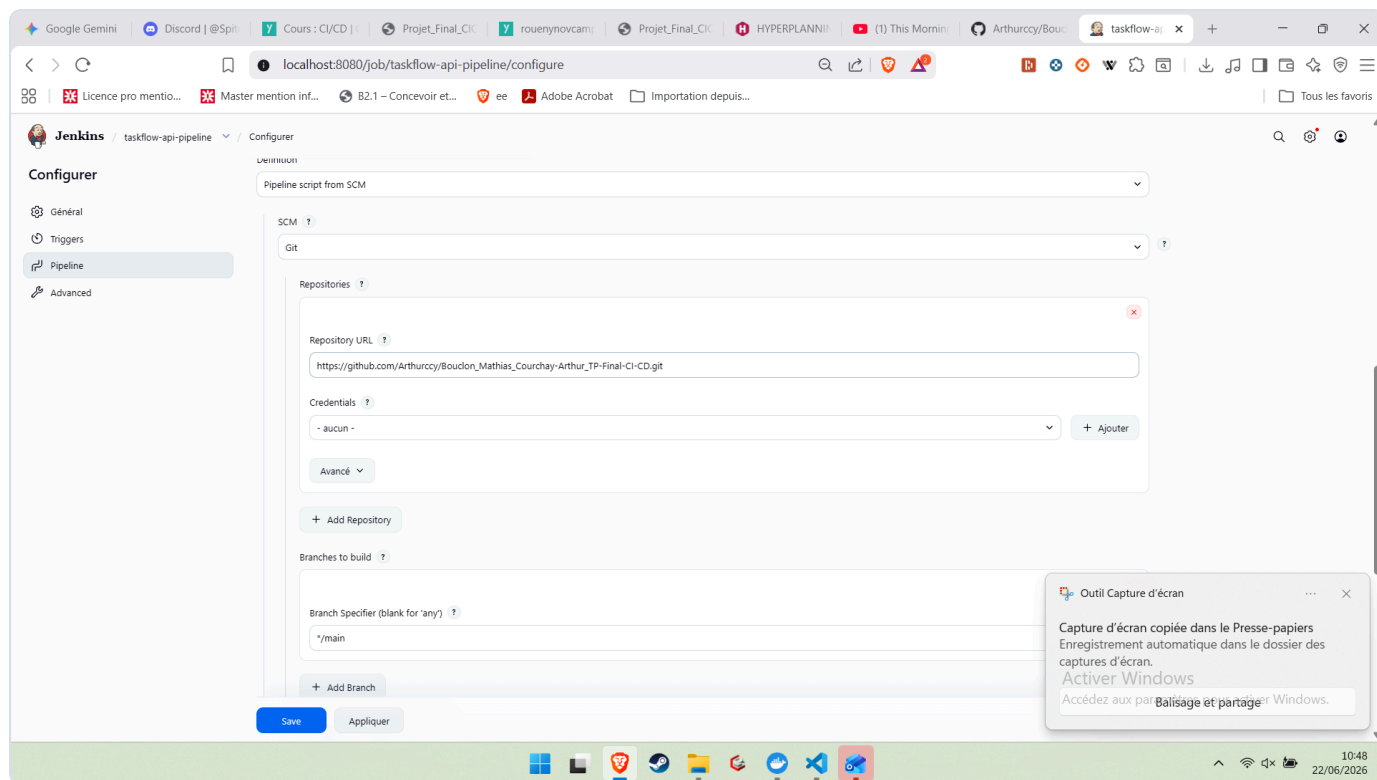


Figure 5 - Stage View Jenkins avec pipeline vert.

7. Configuration Jenkins

Jenkins est lancé comme service Docker et utilise une image dédiée basée sur `jenkins/jenkins:1ts-jdk17`. Cette image installe Docker CLI, Docker Compose, Node/npm et les plugins nécessaires au pipeline.

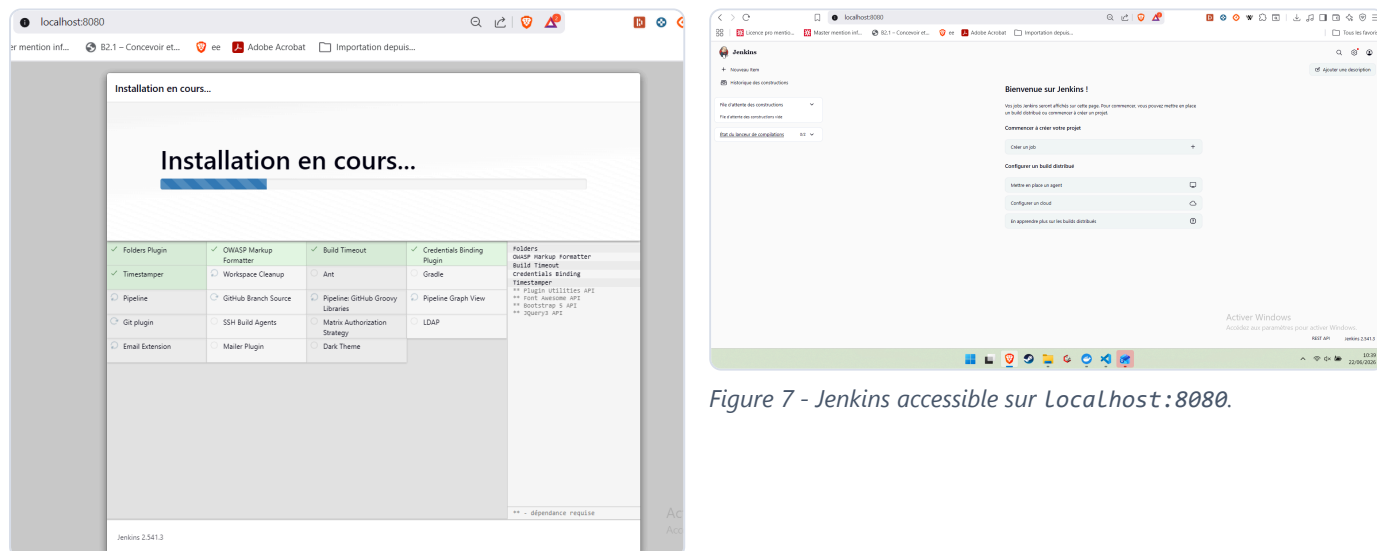


Figure 6 - Installation de Jenkins et plugins.

Figure 7 - Jenkins accessible sur Localhost:8080.

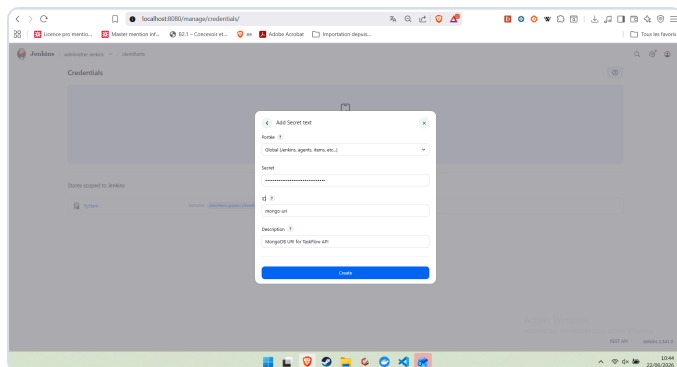


Figure 8 - Credential Jenkins mongo-uri.

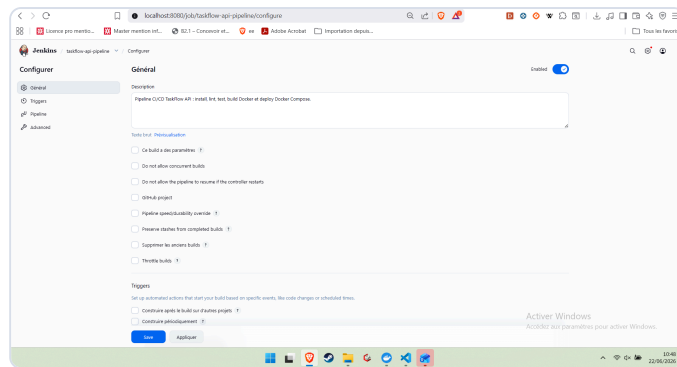


Figure 9 - Configuration du job pipeline depuis GitHub.

8. Preuves de pipeline et déploiement

Le pipeline final exécute les contrôles qualité, construit les images Docker et déploie la stack applicative. Les preuves ci-dessous montrent la réussite des tests et le résultat observable côté Docker.

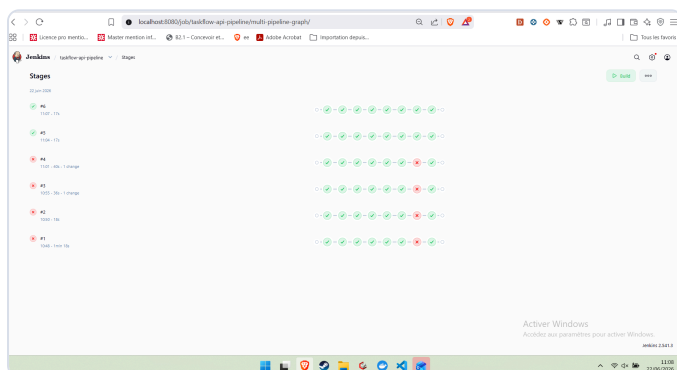


Figure 10 - Vérification des tests et du lint.

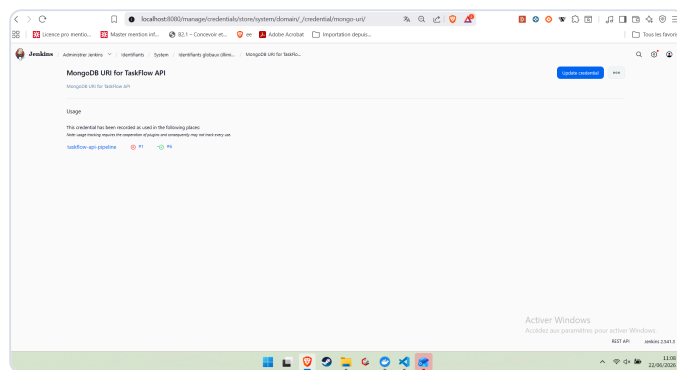


Figure 11 - Validation de la connexion applicative avec MongoDB.

Les endpoints `/health` et `/api/tasks` retournent des réponses JSON valides via Nginx. MongoDB n'est pas exposé à l'extérieur.

9. Build rouge intentionnel

Un build rouge a été provoqué volontairement pour vérifier que le pipeline échoue bien au stage `Lint`. La violation introduite était simple et contrôlée: utilisation de `var` et variable inutilisée. ESLint a bloqué le pipeline, puis un commit correctif a restauré l'état vert.

Étape	Commit	Résultat
Violation volontaire	test: trigger intentional lint failure	Échec au stage <code>Lint</code> .
Correction	fix: restore lint compliance	Pipeline repassé au vert.

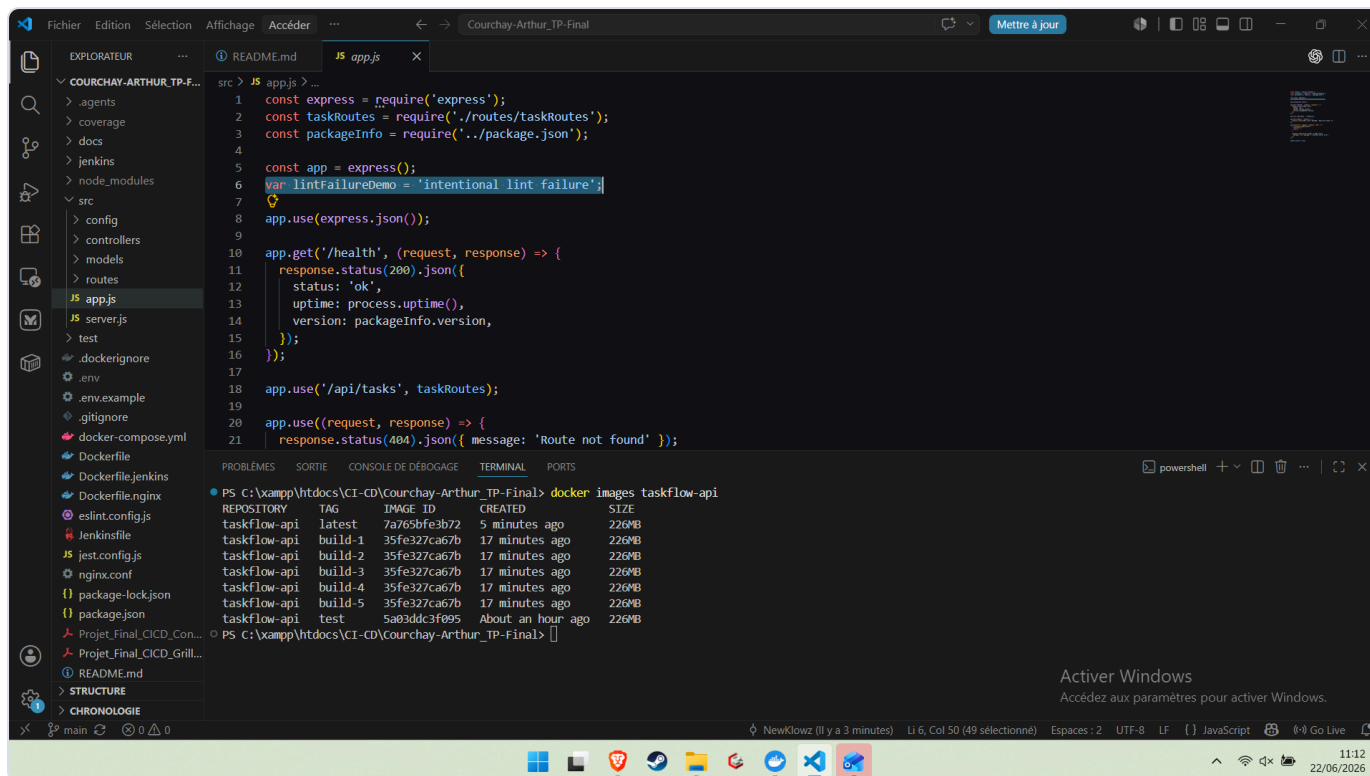


Figure 12 - Build rouge intentionnel au stage Lint.

10. Répartition du binôme

La répartition a été pensée pour rester équilibrée et réaliste: chaque membre a un périmètre principal, mais le binôme a relu et validé l'ensemble du projet afin que chacun puisse l'expliquer.

Membre	Responsabilités principales	Livrables associés
Arthur Courchay	Développement API, modèle Task, validation métier, tests Jest, lint, Dockerfile API.	Routes REST, contrôleurs, tests, couverture, README technique.
Mathias Bouclon	Infrastructure Docker Compose, Nginx, Jenkins, credentials, captures et webhook.	Stack Docker, reverse proxy, configuration Jenkins, preuves du pipeline.
Binôme	Relecture, résolution des blocages, build rouge volontaire, rapport final.	Pipeline vert, documentation, captures, soutenance.

Les deux membres sont capables d'expliquer l'ensemble du flux: requête HTTP, passage par Nginx, API Express, MongoDB, puis chaîne Jenkins jusqu'au déploiement.

11. Difficultés rencontrées et solutions

Difficulté	Symptôme	Solution
Conflit de ports avec un ancien TP	Les ports 80 et 8080 étaient déjà utilisés par ProShop.	Arrêt des conteneurs ProShop avant lancement de TaskFlow.
Jenkins sans Node/npm	Le stage Install ne pouvait pas exécuter <code>npm ci</code> .	Ajout de Node/npm dans <code>Dockerfile.jenkins</code> .
Conflit de noms Docker	Jenkins ne pouvait pas créer <code>taskflow-mongodb</code> déjà présent.	Suppression des <code>container_name</code> fixes et ajout d'un projet Compose stable.
Montage Nginx depuis Jenkins	Le bind mount de <code>nginx.conf</code> échouait depuis le workspace Jenkins.	Création d'une image <code>taskflow-nginx</code> avec la config copiée au build.
Validation du build rouge	Il fallait prouver que le pipeline échoue réellement au lint.	Commit volontaire avec violation ESLint, capture du build rouge, puis correction.

12. Conclusion

Le projet répond aux objectifs du sujet: application créée par le binôme, conteneurisation complète, orchestration avec Docker Compose, reverse proxy Nginx, pipeline Jenkins versionné, tests automatisés et gestion des secrets via credentials. Les captures prouvent à la fois le fonctionnement nominal et les mécanismes de blocage qualité attendus en CI/CD.

TaskFlow API est clonable, compréhensible depuis le README, testable localement et déployable par Jenkins.